

Arboles Balanceados

Teoria Completa

Arboles Binarios de Busqueda (ABB)

Un **arbol binario de busqueda (ABB)** es una estructura jerarquica que organiza los datos de manera ordenada, facilitando la busqueda, insercion y eliminacion. En un ABB, cada nodo cumple las siguientes propiedades:

- Todos los elementos del subarbol izquierdo tienen claves menores que el nodo.
- Todos los elementos del subarbol derecho tienen claves mayores que el nodo.

Los ABB son eficientes para realizar operaciones de busqueda, con una complejidad promedio de $O(\log n)$. Sin embargo, si los datos se insertan en orden creciente o decreciente, el **arbol se desbalancea**, degenerando en una estructura lineal y perdiendo eficiencia.

Arboles AVL

Un **arbol AVL** es un tipo especial de ABB que garantiza el equilibrio en todo momento. Introducido por Adelson-Velsky y Landis, mantiene un **factor de equilibrio (F.E.)** definido como:

$$F.E. = \text{Altura del subarbol izquierdo} - \text{Altura del subarbol derecho}$$

El **F.E.** de cualquier nodo en un AVL siempre esta en el rango de $[-1, 1]$. Si alguna operacion causa un desbalance ($F.E. \notin [-1, 1]$), se realizan **rotaciones** para restaurar el equilibrio.

Tipos de Rotaciones

- **Rotacion Simple:**
 - Izquierda-Izquierda (II): Se aplica una rotacion hacia la derecha.
 - Derecha-Derecha (DD): Se aplica una rotacion hacia la izquierda.

- **Rotacion Doble:**

- Izquierda-Derecha (ID): Combina una rotacion izquierda seguida de una derecha.
- Derecha-Izquierda (DI): Combina una rotacion derecha seguida de una izquierda.

Ventajas de los Arboles AVL

- Garantizan una complejidad de $O(\log n)$ para operaciones de busqueda, insercion y eliminacion.
- Son adecuados para datos desordenados y escenarios en los que el equilibrio es crucial.

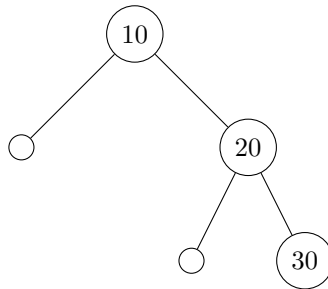
Ejercicios Faciles - Arboles AVL

Ejercicio 1

Inserte las claves {10, 20, 30} y balancee el arbol utilizando una rotacion simple (Izquierda-Izquierda).

Arbol desbalanceado inicial

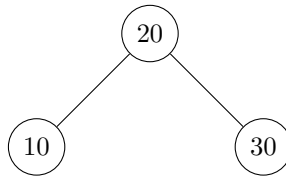
Al insertar las claves en orden, el arbol se forma asi:



El nodo raíz 10 tiene un factor de equilibrio $F.E. = -2$. Esto significa que el subarbol derecho es demasiado alto.

Paso 1: Rotacion Izquierda-Izquierda

Para balancear el arbol, realizamos una rotacion simple hacia la izquierda sobre el nodo 10:



El árbol ahora está balanceado, con todos los nodos teniendo un *F.E.* en el rango $[-1, 1]$.

Código en Java

```
1 class Nodo {
2     int valor;
3     Nodo izquierdo, derecho;
4
5     // Constructor del nodo
6     Nodo(int valor) {
7         this.valor = valor;
8         this.izquierdo = this.derecho = null;
9     }
10 }
11
12 class ArbolAVL {
13
14     // Rotación hacia la derecha
15     Nodo rotacionDerecha(Nodo y) {
16         Nodo x = y.izquierdo;
17         Nodo T2 = x.derecho;
18
19         // Realizar la rotación
20         x.derecho = y;
21         y.izquierdo = T2;
22
23         // Retornar nueva raíz
24         return x;
25     }
26
27     // Método de inserción con balanceo
28     Nodo insertar(Nodo nodo, int valor) {
29         if (nodo == null) {
30             return new Nodo(valor);
31         }
32
33         if (valor < nodo.valor) {
34             nodo.izquierdo = insertar(nodo.izquierdo, valor);
35         } else if (valor > nodo.valor) {
36             nodo.derecho = insertar(nodo.derecho, valor);
37         } else {
38             return nodo; // Valores duplicados no permitidos
39         }
40
41         // Calcular el factor de equilibrio
42         int fe = altura(nodo.izquierdo) - altura(nodo.derecho);
43     }
44 }
```

```

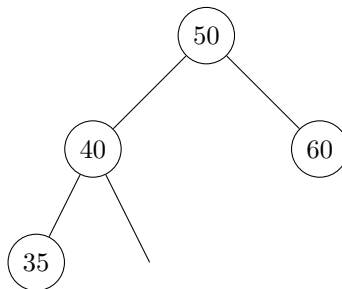
44     // Rotacion II
45     if (fe < -1 && valor > nodo.derecho.valor) {
46         return rotacionIzquierda(nodo);
47     }
48
49     return nodo;
50 }
51
52 // Metodo para calcular la altura de un nodo
53 int altura(Nodo nodo) {
54     if (nodo == null) {
55         return 0;
56     }
57     return 1 + Math.max(altura(nodo.izquierdo), altura(nodo.
58         derecho));
59 }
60
61 // Rotacion hacia la izquierda
62 Nodo rotacionIzquierda(Nodo x) {
63     Nodo y = x.derecho;
64     Nodo T2 = y.izquierdo;
65
66     // Realizar la rotacion
67     y.izquierdo = x;
68     x.derecho = T2;
69
70     // Retornar nueva raiz
71     return y;
72 }

```

Ejercicio 2

Inserte las claves {50, 40, 60, 35} y balancee el arbol utilizando una rotacion simple (Derecha-Derecha).

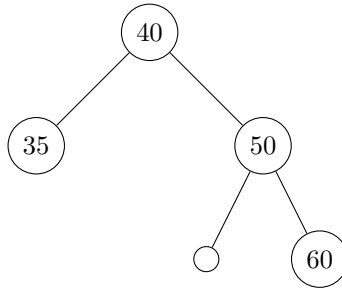
Arbol desbalanceado inicial



El nodo 50 tiene un $F.E. = 2$, lo que indica que el subarbol izquierdo es demasiado alto.

Paso 1: Rotacion Derecha-Derecha

Se realiza una rotacion hacia la derecha sobre el nodo 50:



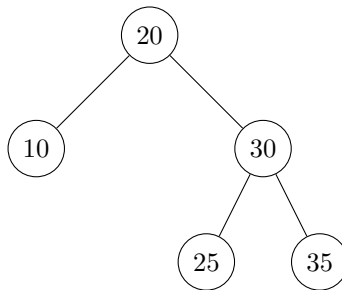
Codigo en Java:

```
1 // Reutilizamos las mismas clases Nodo y ArbolAVL del Ejercicio 1
2 // con la rotacionDerecha implementada
```

Ejercicio 3

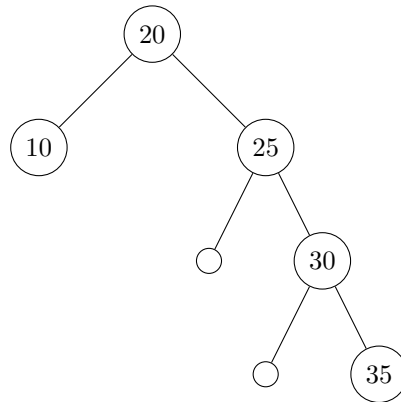
Inserte las claves {20, 10, 30, 25, 35} y balancee el arbol con una rotacion doble (Izquierda-Derecha).

Arbol desbalanceado inicial

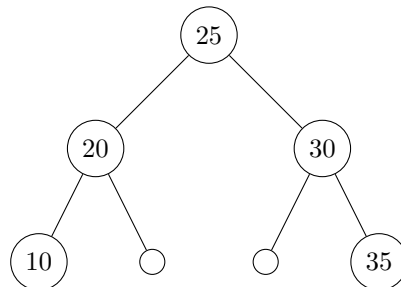


El nodo 20 tiene un $F.E. = -2$, y el nodo 30 tiene un $F.E. = 1$. Esto indica la necesidad de una rotacion doble.

Paso 1: Rotacion hacia la derecha en el subarbol izquierdo de 30



Paso 2: Rotacion hacia la izquierda sobre el nodo 20



El arbol ahora esta balanceado.

Codigo en Java:

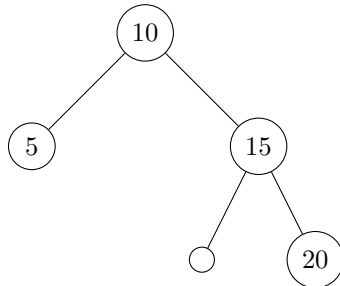
```
1 // Reutilizamos las mismas clases Nodo y ArbolAVL
2 // agregando las llamadas adecuadas a rotacionDerecha y
  rotacionIzquierda
```

Ejercicios Intermedios - Arboles AVL

Ejercicio 4

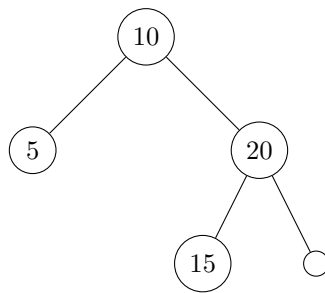
Inserte las claves {10, 5, 15, 20} y balancee el arbol utilizando una rotacion doble (Derecha-Izquierda).

Arbol desbalanceado inicial

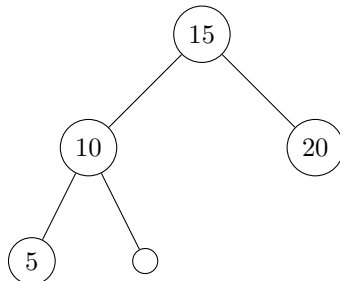


El nodo 10 tiene un factor de equilibrio $F.E. = -2$, y el nodo 15 tiene un $F.E. = 1$. Esto indica la necesidad de una rotacion doble.

Paso 1: Rotacion hacia la izquierda en el subarbol derecho de 15



Paso 2: Rotacion hacia la derecha sobre el nodo 10



El arbol ahora esta balanceado.

Codigo en Java:

```
1 class Nodo {  
2     int valor;  
3     Nodo izquierdo, derecho;  
4  
5     Nodo(int valor) {
```

```

6         this.valor = valor;
7         this.izquierdo = this.derecho = null;
8     }
9 }
10
11 class ArbolAVL {
12
13     Nodo rotacionIzquierda(Nodo x) {
14         Nodo y = x.derecho;
15         Nodo T2 = y.izquierdo;
16
17         y.izquierdo = x;
18         x.derecho = T2;
19
20         return y;
21     }
22
23     Nodo rotacionDerecha(Nodo y) {
24         Nodo x = y.izquierdo;
25         Nodo T2 = x.derecho;
26
27         x.derecho = y;
28         y.izquierdo = T2;
29
30         return x;
31     }
32
33     Nodo insertar(Nodo nodo, int valor) {
34         if (nodo == null) {
35             return new Nodo(valor);
36         }
37
38         if (valor < nodo.valor) {
39             nodo.izquierdo = insertar(nodo.izquierdo, valor);
40         } else if (valor > nodo.valor) {
41             nodo.derecho = insertar(nodo.derecho, valor);
42         } else {
43             return nodo;
44         }
45
46         int fe = altura(nodo.izquierdo) - altura(nodo.derecho);
47
48         if (fe < -1 && valor > nodo.derecho.valor) {
49             if (valor < nodo.derecho.izquierdo.valor) {
50                 nodo.derecho = rotacionDerecha(nodo.derecho);
51             }
52             return rotacionIzquierda(nodo);
53         }
54
55         return nodo;
56     }
57
58     int altura(Nodo nodo) {
59         if (nodo == null) {
60             return 0;
61         }
62         return 1 + Math.max(altura(nodo.izquierdo), altura(nodo.

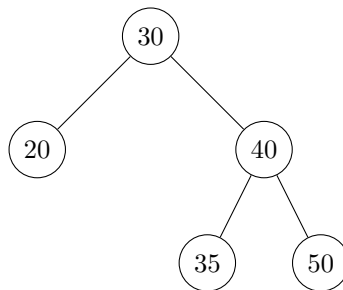
```

```
        derecho));  
63     }  
64 }
```

Ejercicio 5

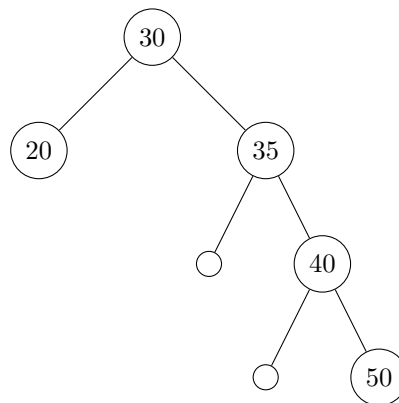
Inserte las claves {30, 20, 40, 35, 50} y balancee el arbol utilizando una rotacion doble (Izquierda-Derecha).

Arbol desbalanceado inicial

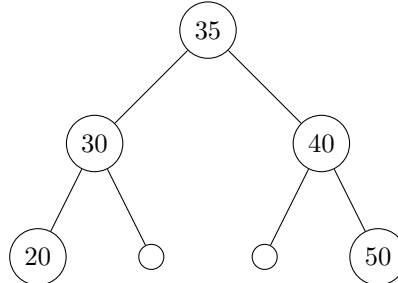


El nodo 30 tiene un factor de equilibrio $F.E. = -2$, y el nodo 40 tiene un $F.E. = 1$. Esto indica la necesidad de una rotacion doble.

Paso 1: Rotacion hacia la derecha en el subarbol izquierdo de 40



Paso 2: Rotacion hacia la izquierda sobre el nodo 30



El arbol ahora esta balanceado.

Codigo en Java:

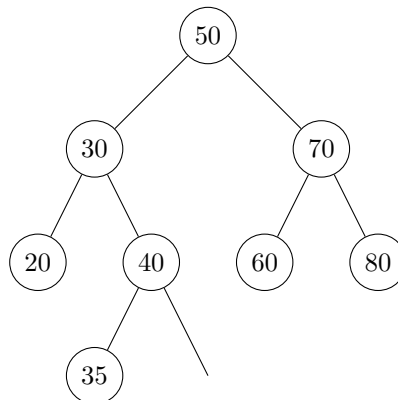
```
1 // El codigo reutiliza las rotaciones rotacionIzquierda y  
  rotacionDerecha  
2 // implementadas en el Ejercicio 4
```

Ejercicios Dificiles - Arboles AVL

Ejercicio 8

Construya un arbol AVL a partir de las claves {50, 30, 70, 20, 40, 60, 80, 35} y balanceelo paso a paso.

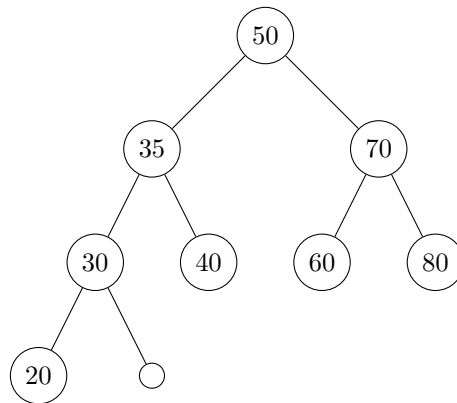
Planteamiento inicial



El nodo 50 tiene un factor de equilibrio $F.E. = -2$. Es necesario balancear el arbol.

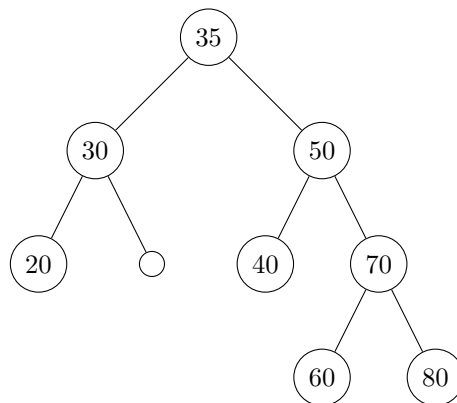
Paso 1: Aplicar rotacion Izquierda-Derecha en el subarbol izquierdo de 50

Primero, aplicamos una rotacion derecha sobre el subarbol izquierdo de 40:



Paso 2: Rotacion hacia la izquierda sobre el nodo 50

Ahora, aplicamos una rotacion izquierda sobre el nodo 50:



El arbol ahora esta balanceado.

Codigo en Java:

```
1 class Nodo {
2     int valor;
3     Nodo izquierdo, derecho;
4
5     Nodo(int valor) {
6         this.valor = valor;
7         this.izquierdo = this.derecho = null;
8     }
9 }
10
11 class ArbolAVL {
12
13     Nodo rotacionIzquierda(Nodo x) {
14         Nodo y = x.derecho;
15         Nodo T2 = y.izquierdo;
```

```

17     y.izquierdo = x;
18     x.derecho = T2;
19
20     return y;
21 }
22
23 Nodo rotacionDerecha(Nodo y) {
24     Nodo x = y.izquierdo;
25     Nodo T2 = x.derecho;
26
27     x.derecho = y;
28     y.izquierdo = T2;
29
30     return x;
31 }
32
33 Nodo insertar(Nodo nodo, int valor) {
34     if (nodo == null) {
35         return new Nodo(valor);
36     }
37
38     if (valor < nodo.valor) {
39         nodo.izquierdo = insertar(nodo.izquierdo, valor);
40     } else if (valor > nodo.valor) {
41         nodo.derecho = insertar(nodo.derecho, valor);
42     } else {
43         return nodo;
44     }
45
46     int fe = altura(nodo.izquierdo) - altura(nodo.derecho);
47
48     if (fe > 1 && valor < nodo.izquierdo.valor) {
49         return rotacionDerecha(nodo);
50     }
51
52     if (fe < -1 && valor > nodo.derecho.valor) {
53         return rotacionIzquierda(nodo);
54     }
55
56     if (fe > 1 && valor > nodo.izquierdo.valor) {
57         nodo.izquierdo = rotacionIzquierda(nodo.izquierdo);
58         return rotacionDerecha(nodo);
59     }
60
61     if (fe < -1 && valor < nodo.derecho.valor) {
62         nodo.derecho = rotacionDerecha(nodo.derecho);
63         return rotacionIzquierda(nodo);
64     }
65
66     return nodo;
67 }
68
69 int altura(Nodo nodo) {
70     if (nodo == null) {
71         return 0;
72     }
73     return 1 + Math.max(altura(nodo.izquierdo), altura(nodo.

```

```
74     }  
75     derecho));
```